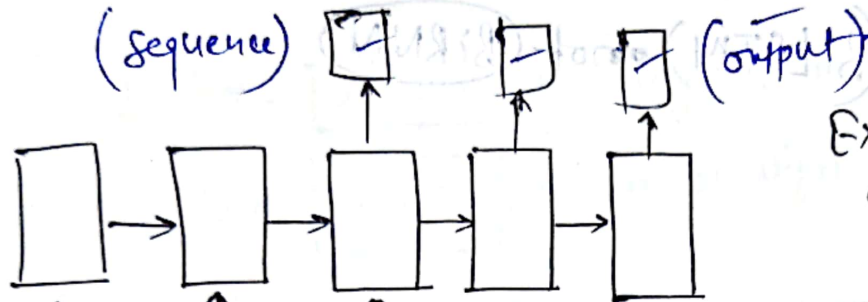
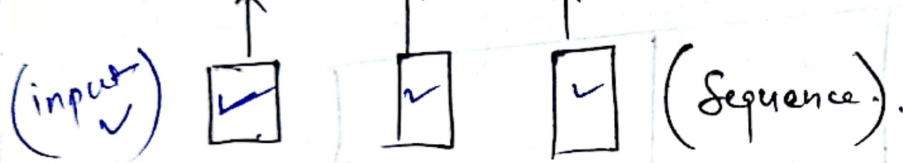


#> LLM's different types of applications

Sequence Tasks & its types :-



Ex:- machine translations
or, language translations.

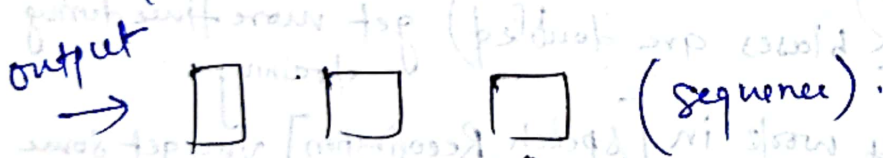


Input & output maybe
diff. or same.

⇒ (Asynchronous) many to many (RNN)

↳ input length may or may not equal to output.

⇒ you accept expects input as sequence or output also sequence.



Parts of speech (POS).

→ (POS) tagging

→ NER. (Name entity Recognition).

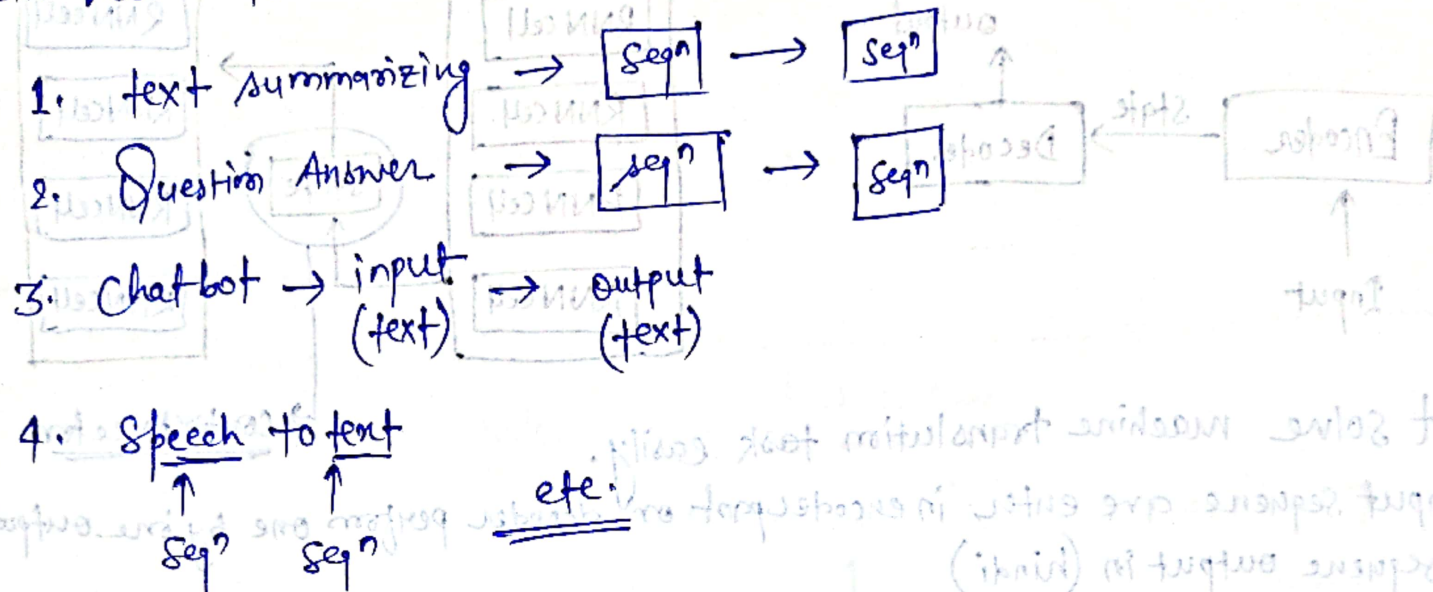


(synchronous). (many to many)

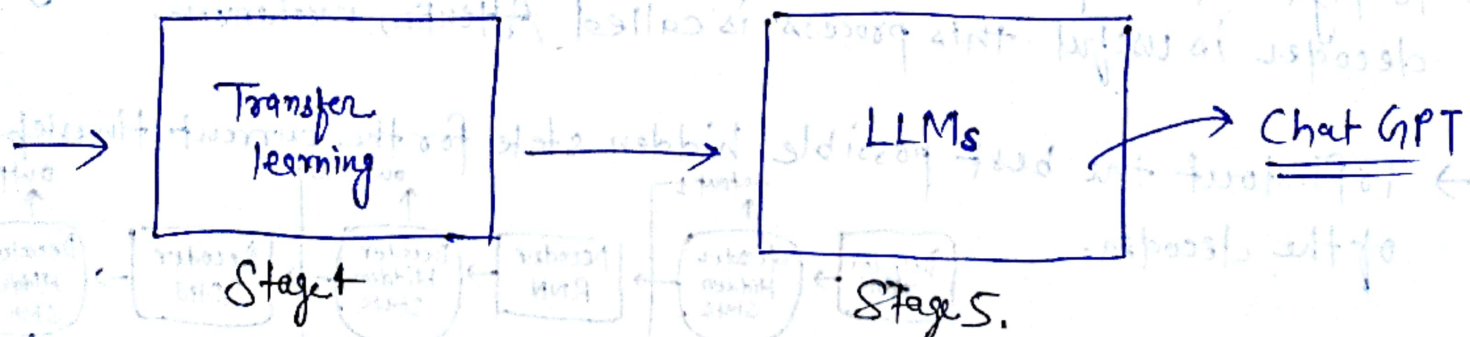
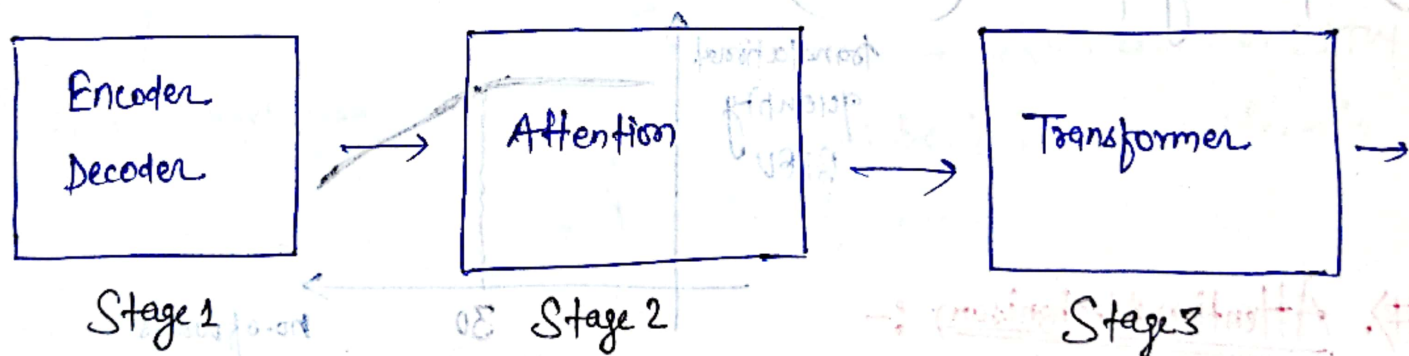
↳ input = output.

#). Sequence 2 sequence Task :-

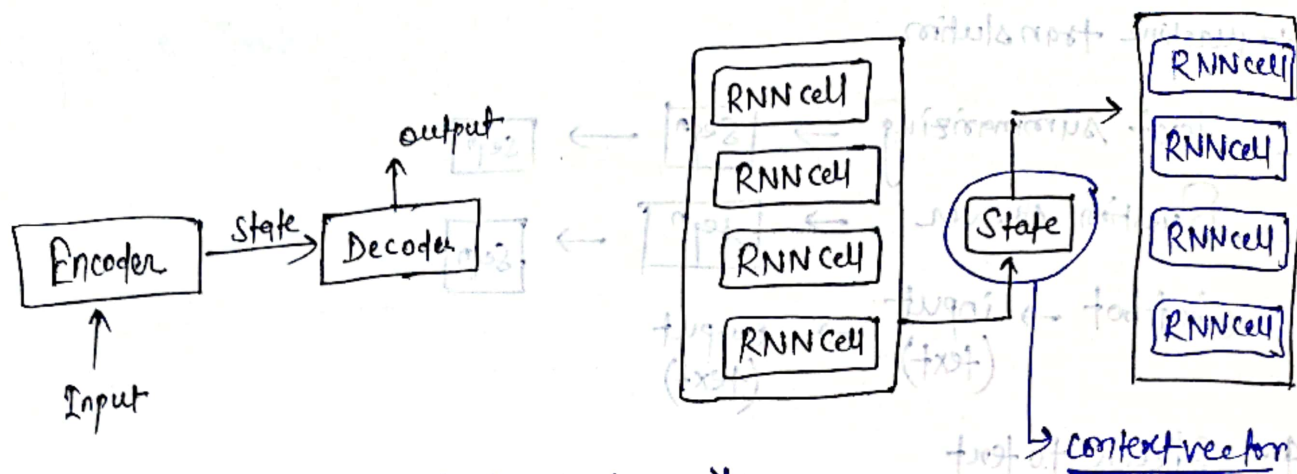
Ex:- machine translation.



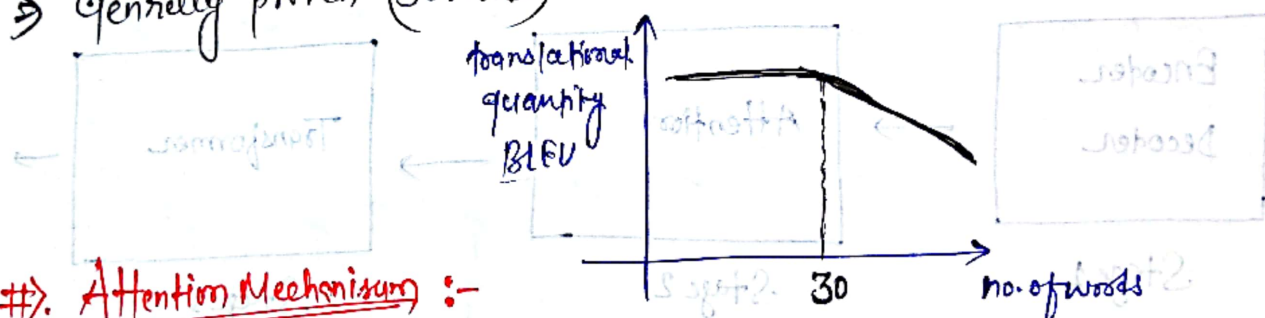
#). History of Seq2Seq Models :-



#> Encoder Decoder Architecture :-



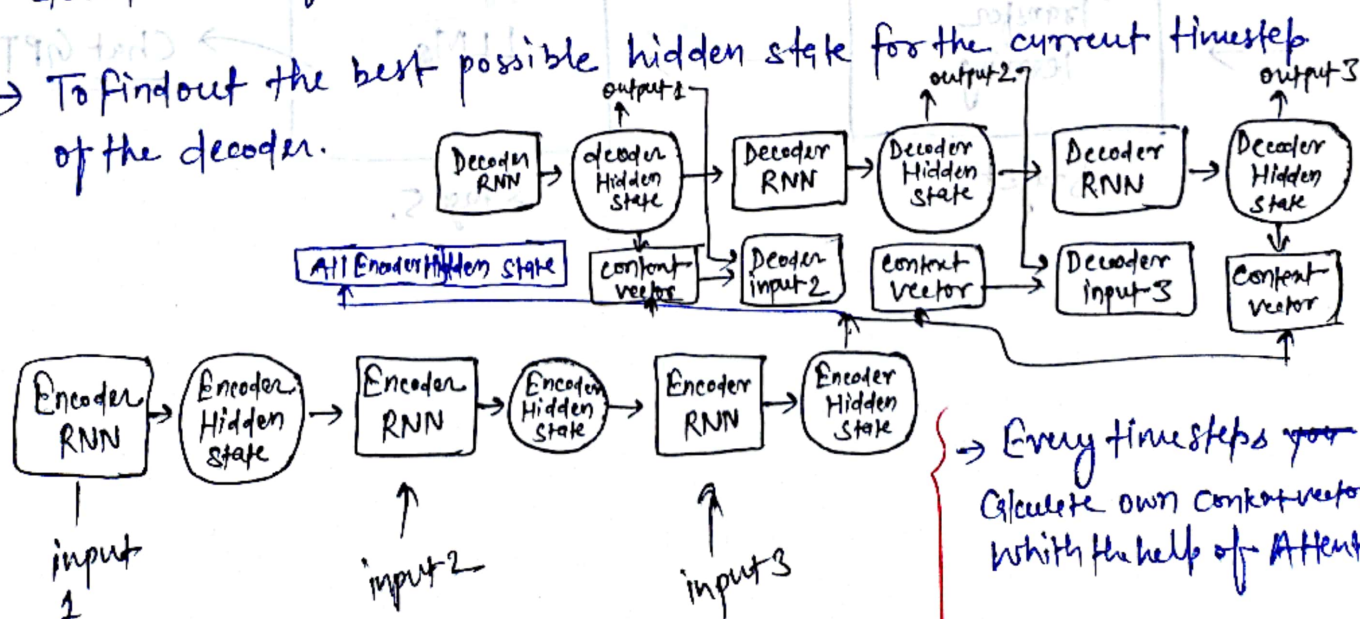
- ⇒ It solve machine translation task easily.
- ⇒ Input sequence are enter in encoder part or decoder perform one by one output sequence output in (hindi)
- ⇒ total summary depends on context vector which is (state)
- ⇒ Problem:- when you have long sequences then decoder doesn't make any sense.
- ⇒ Generally proven (30 words)



#> Attention Mechanism :-

→ to figure out important words into input sentence which is to translate by decoder is useful this process is called Attention mechanism.

→ To Findout the best possible hidden state for the current timestep of the decoder.



→ Every time steps you it calculate own context vector with the help of Attention.

Problems in Attention mechanism :-

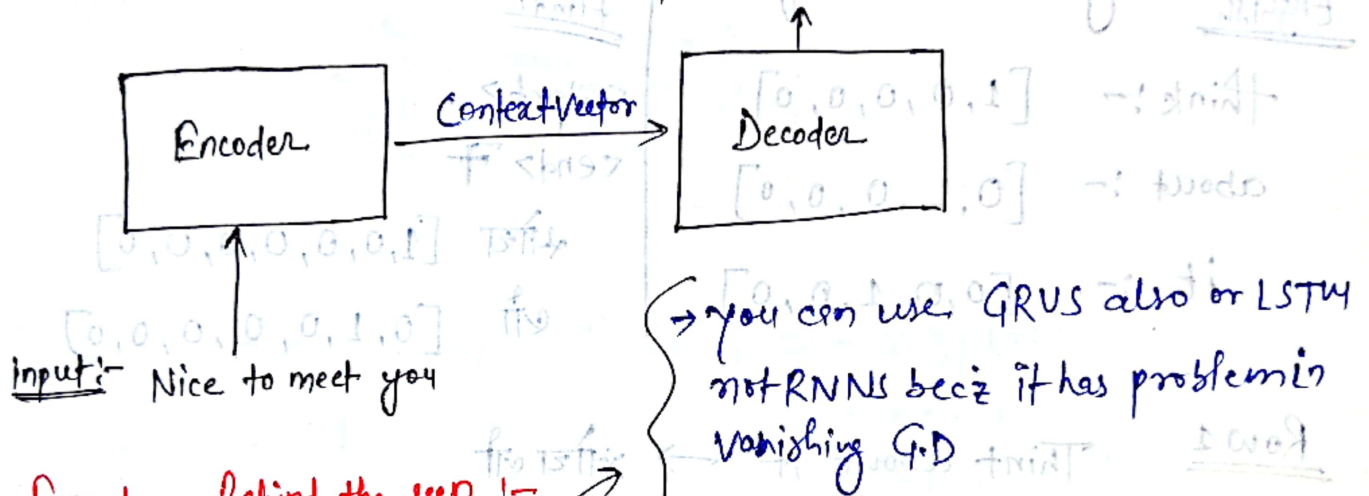
- Computational Complexity
- training time is more than come transformer.

Transformer :-

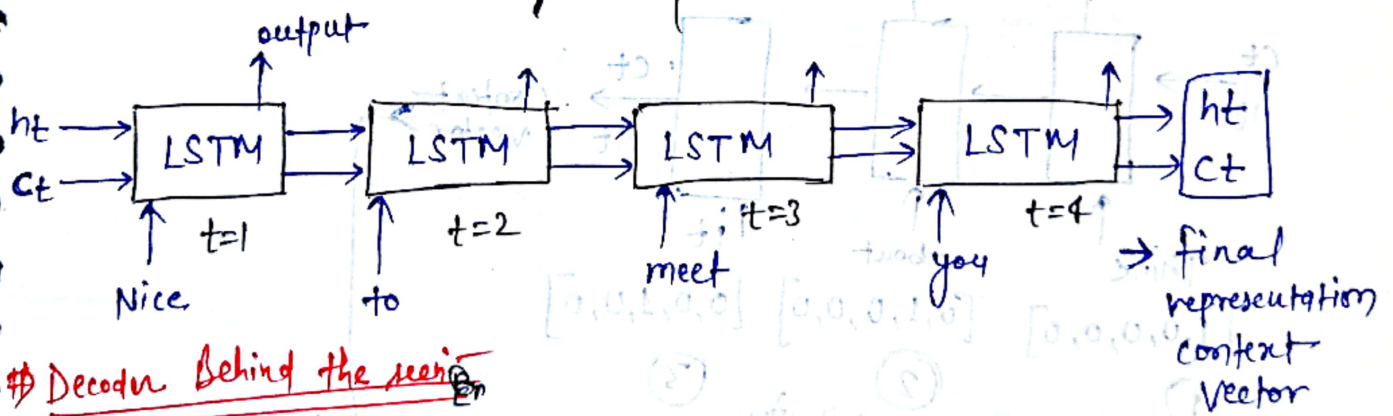
- they remove LSTM/RNN cell or Attention.
- they use self-Attention.

Encoder Decoder Architecture

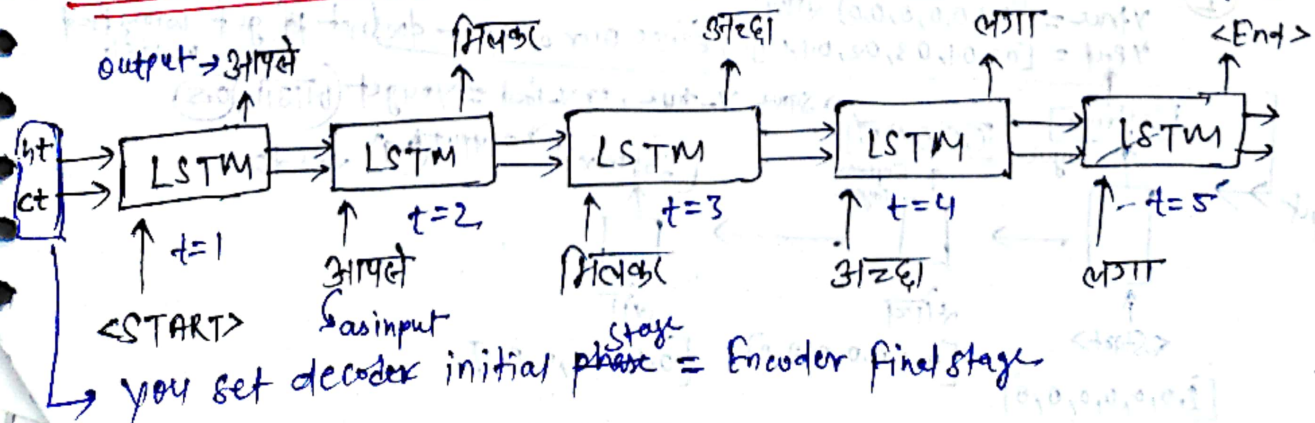
output :- आप से मिल कर अच्छा लगा।



Encoder Behind the scene :-



Decoder Behind the scene



#7. Encoder & Decoder Training the Architecture using backpropagation:-

let's understand with example:-

english	Hindi	Tokenized	
1. Think about it	सोच लो	think, about, it	सोच, लो
2. come in	अंदर आ जाओ	come, in	अंदर, आ, जाओ

Using Onehot encoding

English
think:- $[1, 0, 0, 0, 0]$
about:- $[0, 1, 0, 0, 0]$
it:- $[0, 0, 1, 0, 0]$

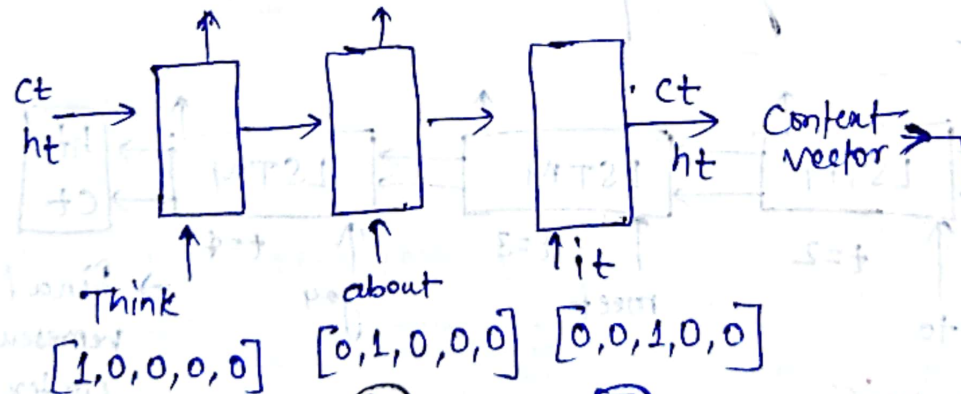
Hindi

<Start>
<end>

सोच $[1, 0, 0, 0, 0, 0, 0]$
लो $[0, 1, 0, 0, 0, 0, 0]$

Row 1

Think about it → सोच लो

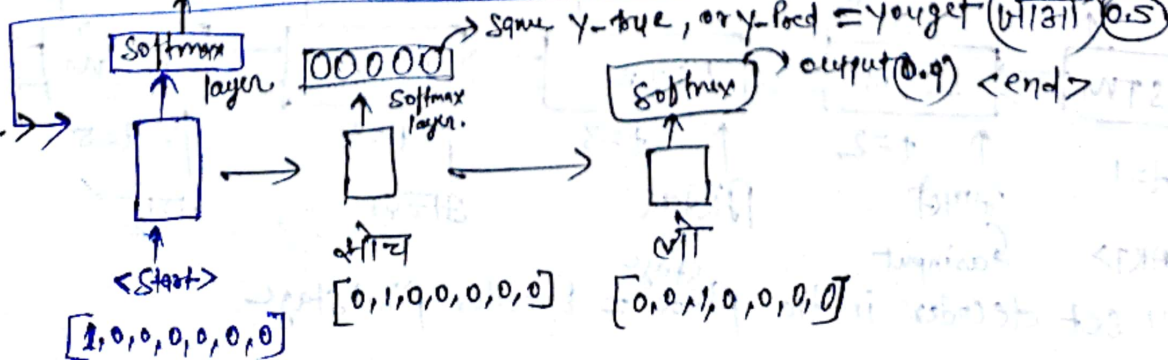


①

$y_{true} = [0, 1, 0, 0, 0, 0, 0]$ सोच
 $y_{pred} = [0.2, 0.1, 0.3, 0.2, 0.1, 0.1]$

②

Since our output doesn't fit get wrong pred model.



⇒ then you calculate loss

loss

$y_{true} = [0, 1, 0, 0, 0, 0]$	लोच	$[0, 0, 1, 0, 0, 0]$	लो	$[0, 0, 0, 0, 0, 1]$	<end>
$y_{pred} = [0.2, 0.1, 0.3, 0.2, 0.1, 0.1]$	लो	$[0.1, 0.2, 0.1, 0.15, 0.3, 0.05]$		$[0.1, 0.2, 0.2, 0.1, 0.3, 0.1]$	
at $t=1$		$t=2$		$t=3$	

pick in loss functⁿ calculate

loss functⁿ = Categorical cross entropy

$$\text{loss} = - \sum_{i=1}^7 y_i^{\text{true}} \log(y_i^{\text{pred}})$$

⇒ $L_{t=1} \Rightarrow -1 \times \log(0.1)$

$L_{t=1} = 1$

⇒ $L_{t=3} \Rightarrow -1 \times \log(0.4)$

$= -1 \times -0.39$
 $= 0.39$

$L_{t=2} \Rightarrow 1$

⇒ then we do back propagation :-

- gradient calculate.
- update parameters.

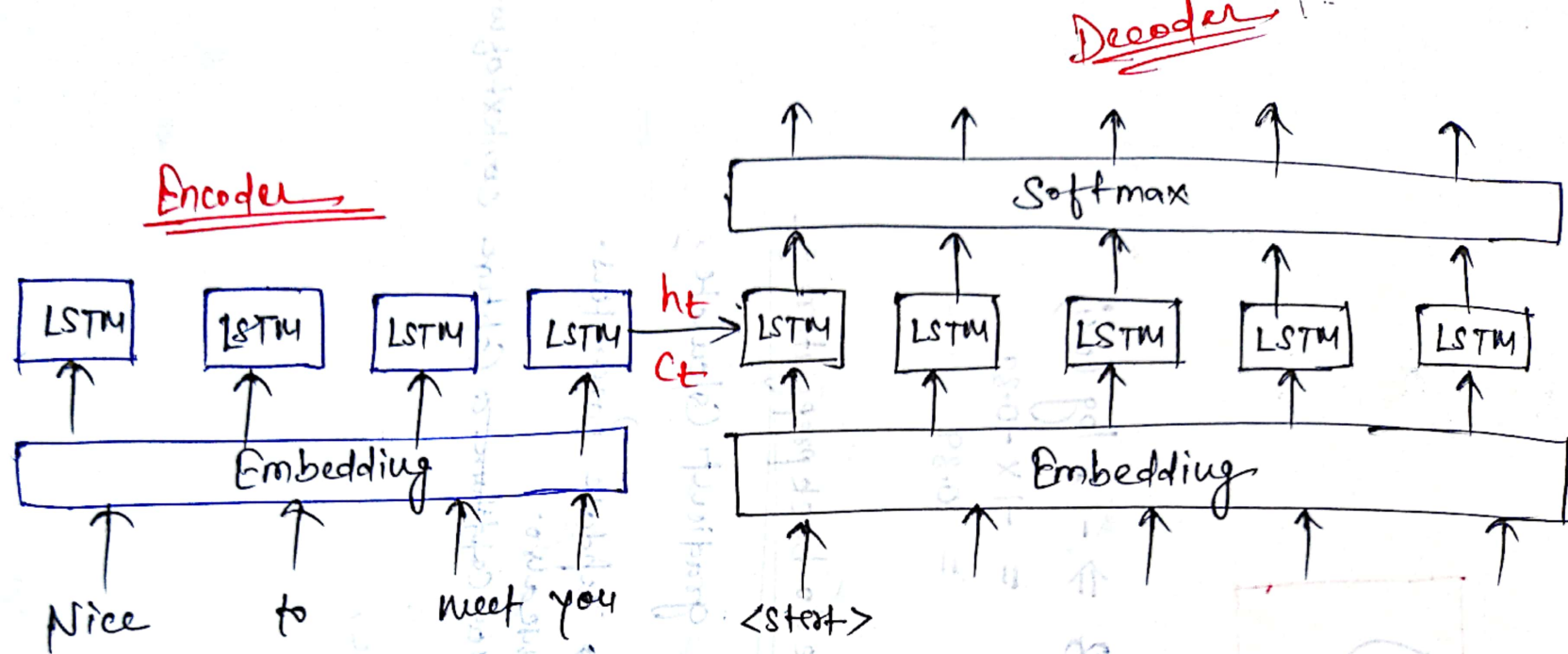
#7. Improvement-1 [Embedding]

→ It provides low dimension. layer or dense also.

→ It is also summary of words. like you capture context of words.

→ Pretrained → word 2 vec / Glove.

→ or you can train.

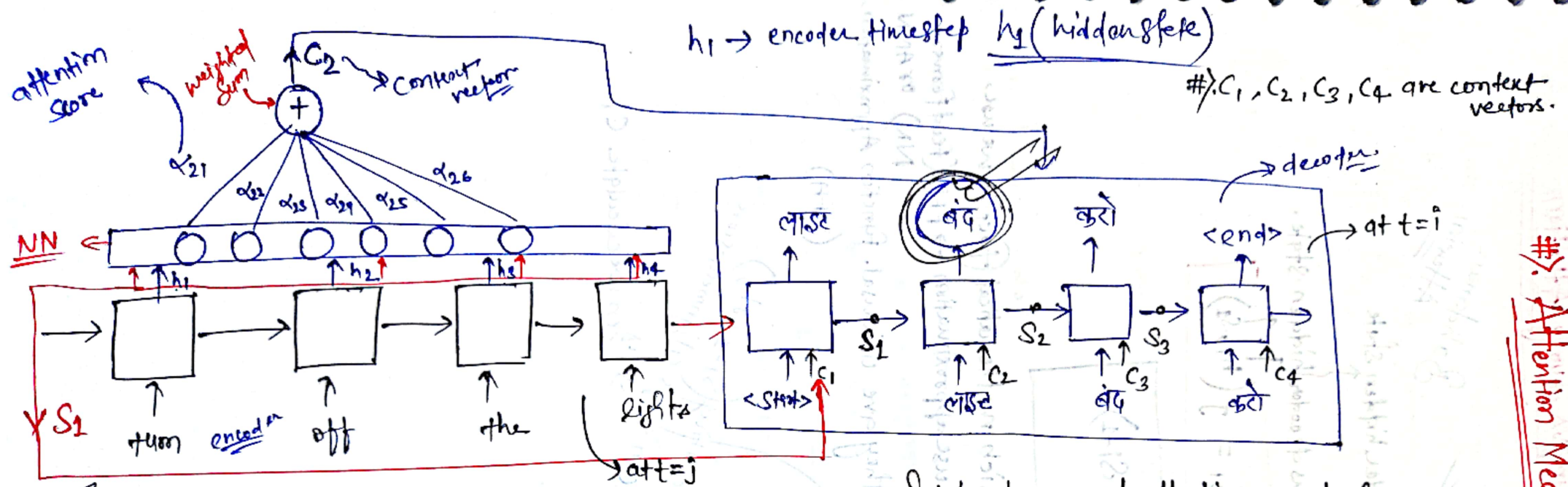


Improvement 2 Deep LSTMs

1. you can handle long term dependency
2. layered representation.

Improvement 3 Reversing the input

- pick word for some language
- for initial context english - french,



$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

BLUE score :- you can improve translation quality

why C_2 $\rightarrow$ context vector for decoder step 2

means Attention weight b/w decoder step 2 and encoder hidden step 1

weighted sum

$$C_i = \sum_{j=1}^4 \alpha_{ij} h_j$$

Q. Why do we need attention mechanism?

Attention helps the model handle long-range dependencies and avoids forcing the entire input sequence into a single fixed size context vector. It allows the model to focus dynamically on relevant information.

How to find (α) attention score / attention score?

$\rightarrow$ we see in next page.

Attention Mechanism

#> Bahdanau Attention vs Luong Attention

#> How to find (α) alignment score

Bahdanau Attention

$$C_i = \sum \alpha_{ij} h_j$$

Encoder hidden state

Previous decoder hidden state.

$\alpha_{11} = ?$

$$\alpha_{11} = f(h_1, s_0)$$

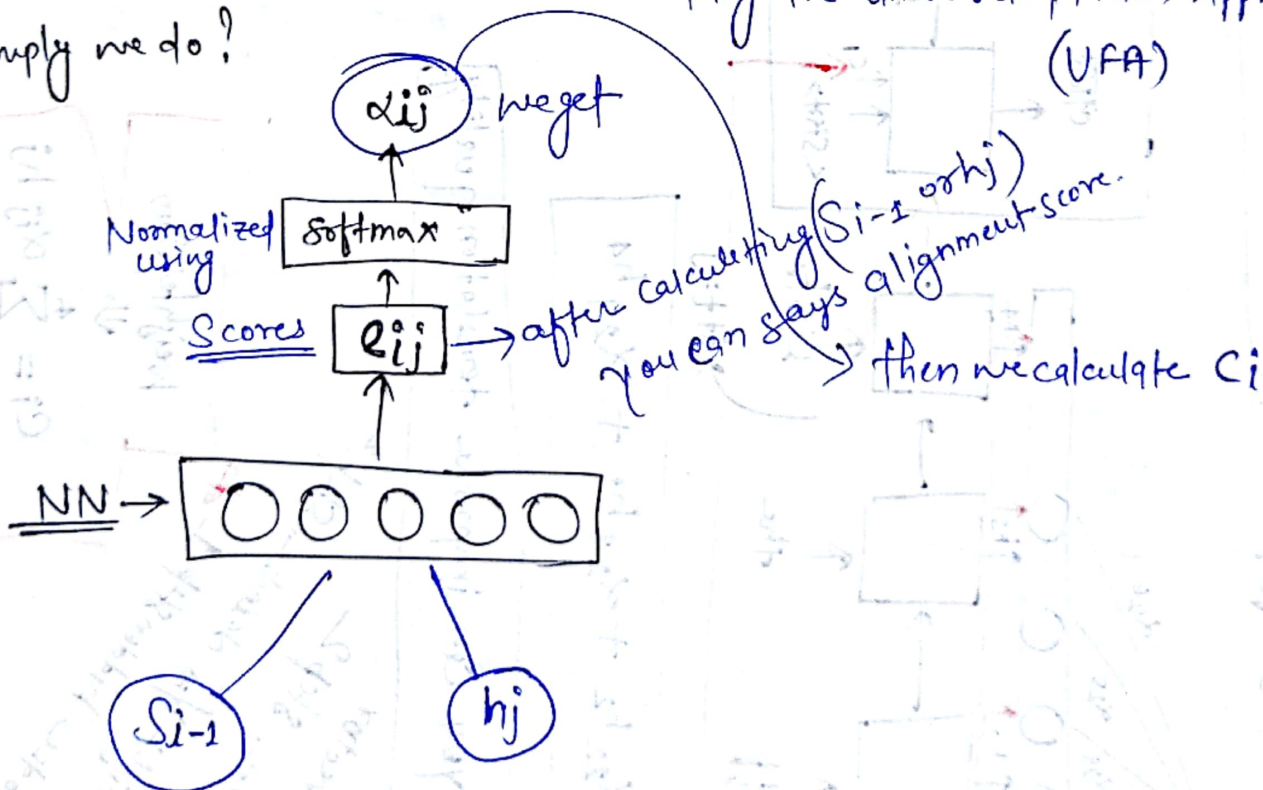
$$\alpha_{21} = f(h_1, s_1)$$

General form :- $\alpha_{ij} = f(h_j, s_{i-1})$

Which mathematical function we use.
we use approximation $\rightarrow$ FFN (Feed Forward NN) or ANN

they are universal function approximation. (UFA)

$\Rightarrow$ Simply we do?



Luong Attention :- also called Multiplicated Attention

In Luong Attention we also calculate C_i

$$C_i = \sum_j \alpha_{ij} h_j$$

In Bahdanau Attention we calculate like $\alpha_{ij} = f(S_{i-1}, h_j)$ or $\alpha_{ij} = f(S_i, h_j)$

but in Luong attention $\Rightarrow \boxed{v \tan(w[S_{i-1}, h_j] + b)}$ or $\boxed{S_i^T \cdot h_j} \rightarrow \text{dot product}$

#> In Luong you do :-

$\rightarrow$ your encoder hidden state & decoder S_0, S_1, S_2 whatever words you have to print then you do dotproduct b/w hidden state & S_0, S_1, S_2 --- and you get ~~S_0, S_1, S_2~~ e_{11}, e_{12}, e_{13} --- then you put in softmax and you get $\alpha_{11}, \alpha_{12}, \alpha_{13}$ --- then you calculating C_1 you put C_1 value with 1st decoder output and you concatenate it $S_1 \parallel C_1$ then you get $\tilde{S}_1$ and you sent this to softmax and you get your output and so on --- (we not sent input content vector and we put in output)